DATA CLEANING AND TRANSFORMATION FOR CLASSIFICATION

Contents

1. Introduction 3
2. Dataset Selection and Profiling 3
3. Data Cleaning Implementation 4
  3.1 Handling Missing Values 4
  3.2 Removing Duplicate Records 4
  3.3 Correcting Inconsistent Labels 4
  3.4 Checking Numerical Values 5
4. Data Transformation 5
  4.1 StandardScaler 5
  4.2 MinMaxScaler 5
  4.3 OneHotEncoder 6
5. Model Development 6
  5.1 Baseline Model 6
  5.2 Improved Model 6
6. Data Leakage Analysis 7
7. Model Evaluation 7
8. Reproducibility and Limitations 8
9. Conclusion 9

1. Introduction

Dataset: Adult Census Income Dataset

For this assignment I chose the Adult Census Income dataset to clean, transform and classify the data. The main goal of this project is to prepare the dataset and build a machine learning model that predicts a person's income category.

Data cleaning is very important, in machine learning. Real-world datasets have missing values, duplicate records and wrong or inconsistent data. If these issues are not fixed they can hurt the results of a machine learning model.

In this project I first looked at the dataset. I spotted its quality problems. Then I tried cleaning techniques. Ran two classification experiments. I also checked for data leakage. I built a preprocessing pipeline to make the experiment more reliable.

2. Dataset Selection and Profiling

The Adult Census Income dataset was obtained from the UCI Machine Learning Repository. It contains information about individuals, including their age, education, occupation, working hours and other personal and employment-related details.

The uploaded dataset has 48,842 rows and 15 columns. It includes six features, eight categorical input features and one target variable.

The numerical features are age, fnlwgt, education-num, capital-gain, capital-loss and hours-per-week. The categorical features include workclass, education, marital-status, occupation, relationship, race, sex and native-country.

The target variable is income. It has two categories: <=50K and >50K. Therefore, the prediction problem is a classification problem. The objective is to predict whether an individual's annual income is than $50,000 using the available features. During the exploration, I used Pandas functions such as head(), shape(), info(), and describe() to understand the structure of the dataset.

I also checked the frequency of values and used histograms to understand the numerical distributions.The age column contains values between 17 and 90, while hours-per-week contains values, between 1 and 99. Capital-gain and capital-loss have zero values and uneven distributionsThe initial inspection also identified values, question-mark symbols, duplicate records and inconsistent income labels. These issues were considered before starting the cleaning process.

Dataset source: https://archive.ics.uci.edu/dataset/2/adult

3. Data Cleaning Implementation

Before making any changes, I created a separate copy of the original dataset. This was done to keep the original data safe and allow the cleaning process to be checked later.

3.1 Handling Missing Values

During the initial inspection, missing values were found in three categorical columns.

- Workclass: 963 missing values.
- Occupation: 966 missing values.
- Native-country: 274 missing values.

I also found question-mark symbols (?) in these columns. These symbols were used to represent unknown information, but Pandas did not automatically identify them as missing values.

Therefore, I replaced the question-mark symbols with proper NaN values. After this step, the total missing values were 2,795 in workclass, 2,805 in occupation, and 856 in native-country.

For model development, I used median imputation for numerical features and most-frequent imputation for categorical features. Median imputation is useful when numerical data contains unusual values, while most-frequent imputation replaces missing categorical values with the most common category.

3.2 Removing Duplicate Records

I checked the dataset for records, with the duplicated() function. The first look found 29 duplicate records.

I removed those records using the drop_duplicates() function. After the removal the dataset has 48,790 rows and 15 columns.

I removed the duplicate records because keeping observations could give repeated information that might unnecessarily influence the model training.

3.3 Correcting Inconsistent Labels

Another problem was discovered in the income column. Some records had labels such as <=50K. And > 50 K whereas other records lacked the period.

These labels represented the two income categories. The unnecessary final period was. The target values were standardized.

After cleaning the target variable held 37,109 records in the <50K category and 11,681 records, in the >50K category.

Unnecessary leading and trailing spaces were also removed from the values to make the category labels consistent.I also removed unnecessary leading and trailing spaces from categorical values to make the category labels consistent.

3.4 Checking Numerical Values

I used numerical summaries to check the minimum and maximum values of the numerical columns. I also checked whether negative values were present.

No negative values were identified in the numerical features. I did not remove unusual values without a proper reason because some extreme values can be valid observations.

For example, a high capital-gain value does not automatically mean that the record is incorrect.

After these cleaning steps, the working dataset was ready for the transformation and modelling stages.

4. Data Transformation

After cleaning the dataset, I considered different methods to transform the numerical and categorical features.

4.1 StandardScaler

StandardScaler was used to standardise numerical values. It transforms the values based on the mean and standard deviation.

This method is useful for Logistic Regression because the numerical features have different ranges. For example, age and capital gain have very different numerical values.

4.2 MinMaxScaler

MinMaxScaler was also tested to compare its effect with StandardScaler. It transforms numerical values into a range between 0 and 1.

This makes it easier to compare features with different measurement ranges. However, this method can be affected by extreme values.

After comparing the two methods using the age feature, StandardScaler was selected for the improved model.

4.3 OneHotEncoder

OneHotEncoder was used to convert categorical values into numerical features.

For example, categories such as Male and Female are represented using separate binary columns. This method was selected because categorical values should not be given an artificial numerical order.

The encoder was configured to ignore unknown categories during testing.

All final preprocessing steps were included in Scikit-learn pipelines so that they could be fitted using training data only.

5. Model Development

For the classification experiments Logistic Regression was selected. Logistic Regression is an used classification algorithm and is suitable for predicting between two classes. Two experiments were developed to compare the effect of scaling. The dataset was divided into training and testing sets using an 80:20 ratio. Random_state=42 was used to make the classification experiments reproducible and stratified splitting was used to maintain the target class proportions. The training dataset contained 39,032 observations and the testing dataset contained 9,758 observations.

5.1 Baseline Model

The baseline model used median imputation for numerical features and most-frequent imputation followed by OneHotEncoder for categorical features.

Numerical scaling was not included in this experiment.

5.2 Improved Model

The improved model used the same imputation methods and categorical encoding as the baseline model. However, StandardScaler was added to the numerical preprocessing steps.

Both models used Logistic Regression and the same train/test split. This allowed me to compare their results under the same experimental conditions.

6. Data Leakage Analysis

Data leakage is a problem that can happen when information from the testing dataset is used during the training or preprocessing stage.

For example, fitting a scaler or imputer using the complete dataset before splitting it into training and testing data can introduce information from the test set into the preprocessing process.

To demonstrate this problem, I created a flawed example in which preprocessing was performed before the train/test split.

However, this example was used only to demonstrate an incorrect approach and was not considered a valid model result.

For the corrected approach, I first divided the dataset into training and testing sets. Then I used Scikit-learn pipelines to perform imputation, scaling, and encoding.

In this way, preprocessing methods were fitted using only the training data and then applied to the testing data.

The baseline and improved experiments followed this corrected approach. Therefore, their evaluation results were used for the final comparison.

7. Model Evaluation

After training both models, I evaluated their performance using accuracy, precision, recall, F1-score, and ROC-AUC.

The following table shows the results obtained from the experiments.

Evaluation metric	Baseline	Improved
Accuracy	0.8498	0.8520
Precision	0.7309	0.7304
Recall	0.5895	0.6053
F1-score	0.6526	0.6620
ROC-AUC	0.8874	0.9081

The improved model achieved slightly better results in all five evaluation metrics.

Accuracy measures the percentage of records that were classified correctly. Precision tells how many of the records predicted to be income turned out to be higher income. Recall tells how many of the higher income records were correctly identified.

F1-score provides a balance between precision and recall, while ROC-AUC measures how well the model can distinguish between the two income classes.

The dataset contains records in the <=50K category than, in the >50K category. Because of this class imbalance, accuracy alone is not enough to evaluate the model. Precision, recall and F1-score are also important.

The confusion matrix of the improved model showed that 6,900 lower-income records and 1,414 higher-income records were correctly classified. However 522 lower-income records were incorrectly predicted as higher-income and 922 higher-income records were incorrectly predicted as income.

These results show that the model can identify both income categories. Some incorrect predictions still occur.

8. Reproducibility and Limitations

Reproducibility is important because another person should be able to follow the same steps and obtain comparable results.

In this project, I used a fixed random seed of 42 and a stratified 80:20 train/test split. I also documented the dataset source, cleaning methods, transformation methods, and classifier settings.

The Python, Pandas, NumPy, and Scikit-learn versions should be recorded from the actual execution environment. The original CSV file, cleaned dataset, model comparison results, and Jupyter Notebook should also be kept as supporting files.

There are some limitations in this project. Only Logistic Regression was used for the two experiments. Other algorithms such as Decision Tree and Random Forest could be tested in future work.

Also, the experiment used one train/test partition. Cross-validation could provide a more complete understanding of model performance.

Another limitation is that the dataset contains historical demographic and employment information. Therefore, the predictions may reflect patterns and biases present in the original data.

The improved model also showed only a small improvement over the baseline model in accuracy. This means that numerical scaling helped the model in this experiment, but it does not guarantee better performance in every dataset or situation.

9. Conclusion

Through this assignment, I gained hands-on experience in looking at a dataset and getting it ready for machine learning. I found issues, with the data like missing values, repeated entries and labels that didn't match in the categories. I also learned how to use StandardScaler, MinMaxScaler and OneHotEncoder to change the data. By creating two Logistic Regression experiments, I was able to see how a simple way of preparing the data compared to a way that included scaling numbers. The better method gave results for the evaluation metrics I chose. I also realized how important it is to stop data leakage by making sure the preparation steps only used the training data. Overall, this assignment showed me that cleaning data and getting it ready is a part of making a good machine learning model. It also made me see how important it is to write down every choice I made and to make sure the experiments can be done again.